

Fetch: Linking variables between import steps

Contents

- [Contents](#)
- [Why](#)
- [Propagated data](#)
- [Query syntax](#)
- [UUID binding](#)
 - [Example](#)

Why

When importing multiple layers of information, the root- and children queries will probably not contain enough information to link data imported in consecutive steps. There could even be multiple records per step, per node from the initial roots query. An object can have more than one aspect, for instance. Fetch provides a mechanism to work with data from the previous steps.

Propagated data

The Fetch add-on propagates all data it recognizes as URI's or ids. To determine what to save for each import step, the following patterns are looked for:

Pattern	Example	Example usage
<code>BIND (UUID() as ?parametername)</code>	<code>BIND (UUID() as ?objectUuid)</code>	<code>values (?imported_objectUuid) { ?placeholder }</code>
<code>BIND (?<any prefix>Uri as ?parametername)</code>	<code>BIND (?aspectUri as ?attributeClassExternalId)</code>	<code>values (?imported_attributeClassExternalId) { ?placeholder }</code>
<code>BIND (?<any prefix>Uuid as ?parametername)</code>	<code>BIND (?objectUuid as ?attributeClass)</code>	<code>values (?imported_attributeClass) { ?placeholder }</code>

All above patterns are matched using case insensitive regular expressions, so `uuid()` or `?someURI` would also be matched. Only variables that are bound to a variable that is actually returned by the query, will be propagated.

Query syntax

To use these propagated values in a query in a later step, the use of this specific syntax is required:

```
values (?imported_variable1 ?imported_variable2) {
  ?some_placeholder
}
```

The name of the variables needs to match the variable name of one of the patterns matched above, prefixed with "imported_". The name of the placeholder inside the values block is free to choose. The whole values block is matched by a regular expression, which will automatically create a named group for the placeholder. The code doesn't work with a predefined value, but rather uses the matched value to do the variable substitution.

UUID binding

To avoid having to rebuild the entire graph to get to nested levels, the propagated values are internally grouped by UUID. The business logic uses the UUID bound to ?uuid as an identifier to store the right data to be propagated. Therefore it is required to have a `BIND(?uuid AS ?someVariable)` in every query from which you want to propagate values.

Example

Let's assume we're importing Autobaan and Brugpijler from the object hierarchy below. Autobaan has aspects breedte and capaciteit, Brugpijler has aspects breedt, hoogte and lengte.



Importing objects, aspects and possessed aspects in a decomposition hierarchy into BIM Portal requires splitting up the process into 6 import steps:

1. Import objects
2. Link objects using decomposition relations
3. Import attribute classes (aspects)

4. Import attributes (posessed aspects)

```
5. SELECT  ?objectUuid ?attributeExternalId ?attributeClassExternalId
           ?value ?unitOfMeasurement ?unitOfMeasurementUri ?name
           ?minimumCardinality ?maximumCardinality
6. WHERE {
7.   BIND(UUID() as ?attributeExternalId)
8.   BIND (?uuid as ?objectUuid)
9.   BIND (?uri as ?objectUri)
10.  BIND (?aspectUri as ?attributeClassExternalId)
11.
12.  ?aspectRelation otl:rel-Relation-toRole ?posessedAspect ;
13.                  otl:rel-Relation-from ?objectUri ;
14.                  otl:rel-Relation-to ?aspectUri .
15.  ... assemble more attribute details
}
```

16. Link attribute classes to attributes

```
17. SELECT ?sourceId ?targetId ?relationType
18. {
19.   BIND(?uuid as ?objectUuid)
20.   BIND(?imported_attributeExternalId as ?targetId)
21.   BIND(?imported_attributeClassExternalId as ?sourceId)
22.   BIND("HAS_ATTRIBUTE_CLASS" as ?relationType)
23.
24.   VALUES (?imported_attributeExternalId
              ?imported_attributeClassExternalId) {
25.     ?imported_placeholder
26.   }
}
```

27. Link attributes to physical objects

```
28. SELECT  ?sourceId ?targetId ?relationType
29. {
30.   BIND(?uuid as ?sourceId)
31.   BIND(?imported_attributeExternalId as ?targetId)
32.   BIND("HAS_PROPERTY" as ?relationType)
33.   VALUES (?imported_attributeExternalId) {
34.     ?imported_placeholder
35.   }
36. }
```

Steps 1-4 are entirely driven by data from the selected objects in the tree: object uri's and UUID's provide a convenient entry point for queries. In query 4, for collecting the attributes, we generate a new UUID to serve as unique key in the BIM Portal. We also set aside the URI of the attribute class in that query.

For query 5 however, we need to know which attributes should be linked to which attribute class. We do this by preparing the sparql queries so that all pairs of attribute - attribute class are added to the VALUES clause and bound to sourceId and targetId.

For query 6 we need the link between the object and the attribute. After query 4 we stored the combinations of object UUID's with their matching attribute ids. For example attribute UUID's for aspects of Autobaan would be stored linked to Autobaan's UUID. The ?uuid parameter used in the sparql query is always filled using the UUID's from the selected node from the add-on UI. Unfortunately, in this import step we have no way of getting to the UUID generated for the attribute in step 4. We need to get it from our propagated values by the UUID of the selected object.